

Atos Origin
WORLDWIDE IT PARTNER

A la redécouverte de JavaScript

Raphaël Semeteys (raphael.semeteys@atosorigin.com)
Consultant Open Source

Solutions Linux 2008

© 2008 Atos Origin. Document exclusivement réservé à usage commercial. Tout ou partie de ce document ne peut être copié, modifié, diffusé ou annoté sans accord écrit d'Atos Origin ou du client.

Sommaire

Introduction

ECMAScript

Aspects techniques méconnus

Outils libres disponibles

Le futur de JavaScript

Introduction

JavaScript
- Langage de script, orienté objet,
- Inspiré de Self, sans lien avec Java

Qui a souffert de la guerre des navigateurs
- Créé par Netscape, implémenté par Microsoft
- DOM incompatibles

Mais grâce au Web 2.0, il a de nouveau le vent en poupe

Cependant, il reste souvent peu (re)connu
- Déconsidéré par les développeurs coté serveur
- Certaines de ses particularités sont ignorées

ECMAScript

JavaScript ou ECMAScript ?

ECMAScript
- Spécification de l'organisme de standardisation "Ecma International" en 1997
- ECMAScript (ECMA-262) : correspond à JavaScript Core (hors browser et DOM)
- Composants ECMAScript (ECMA-290) : aucune implémentation connue
- E4X (ECMA-357) : support de XML

Les technologies JavaScript et JScript visent désormais l'implémentation du standard tout en proposant de nouvelles fonctionnalités pas (encore) incluses dans ECMAScript.

Il existe d'autres implémentations

ECMAScript

Implémentations ECMAScript

Implémentation	Éditeur	Langage	Licence	Remarques
SpiderMonkey	Mozilla	C	MPL/GPL/LGPL	Première implémentation en 1995 Intégré à Firefox
JScript	Microsoft	C++	Propriétaire	Intégré en 1996 à Internet Explorer
JScript .Net	Microsoft	C++	Propriétaire	Version utilisable uniquement coté serveur
Rhino	Mozilla	Java	MPL/GPL/LGPL	Existe depuis 1998, intégré à Java SE 6 depuis décembre 2006
Presto	Opera	?	Propriétaire	A remplacé Elektra en 2003
ActionScript	Adobe	C++	Propriétaire	Tamarin
KDE's JavaScript engine (KJS)	KDE	C++	GPL	Intégré à Konqueror en 2000
JavaScriptCore	Apple	C++	GPL	Basé sur KJS en 2002 et intégré à WebKit Sélectionné pour Android

Aspects techniques méconnus

Orientation objet – Objets génériques

Dans JavaScript tout est objet ou référence à un objet

Objet génériques

```
var MonObjet = new Object;
MonObjet.attribut = "Attribut";
MonObjet.fonction = function() {
    return "Fonction";
}

MonObjet.attribut;       //retourne "Attribut"
MonObjet.fonction() ;    //retourne "Fonction"
```

Peut être vu comme un tableau associatif

```
MonObjet['attribut'];    //retourne "Attribut" (équivalent à MonObjet.attribut)
MonObjet['fonction']; :  //retourne : function () { return "Fonction"; }
MonObjet['fonction']();  //retourne "Fonction" (équivalent à MonObjet.fonction())
```

Déclaration simplifiée

```
var MonObjet = { attribut: 'Attribut' , fonction: function() { return 'Fonction'; } };
```

Aspects techniques méconnus

Orientation objet – Constructeurs

Constructeur

- Permet de créer des objets plus complexes
- Fonction exécutée à la création de l'objet (via l'instruction new)
- Accessible via la propriété “constructor'

Objet génériques

```
function Voiture(param) {
    this.marque = param;
    this.demarrer = function() {
        return "vroom !";
    }
}

var maVoiture = new Voiture("Renault");
maVoiture.marque;        //contient "Renault"
maVoiture.demarrer();    //retourne "vroom !"
```

this : dans le corps du constructeur, fait référence à l'objet en cours de création

Aspects techniques méconnus

Orientation objet – Prototypes (1/2)

Prototype
- Objet associé à toute fonction pour référence par les objets créés
- Accessibe via la propriété “prototype” du constructeur

Mécanisme de remontée des prototypes
- Affectation d'une propriété non existante : créée au niveau de l'objet
- Récupération de la valeur d'une propriété (existante ou pas)
  - si elle existe au niveau de l'objet, sa valeur est retournée
  - sinon, recherche au niveau du prototype de l'objet
  - sinon, recherche au niveau du prototype du prototype
  - etc...
  - sinon, retourne “undefined”

Aspects techniques méconnus

Orientation objet – Prototypes (2/2)

Permet de modifier dynamiquement les objets créés avec le même constructeur

```
function Voiture(param) {
    this.marque = param;
}
var maVoiture = new Voiture("Relault");

Chien.prototype.demarrer = function() {
    return "Vroom !";
}

maVoiture.demarrer(); //retourne "Vroom !"
```

Egalement valable pour les objets prédéfinis de JavaScript

```
String.prototype.inverser = function() {
    var inverse = "";
    for (i = this.length - 1; i >= 0; i--) {
        inverse += this.charAt(i);
    }
    return inverse;
}
var chaine = "Bonjour !";

chaine.inverser(); //retourne "! ruojnoB"
```

Aspects techniques méconnus

Orientation objet – Héritage par prototypes

Héritage par assignation de l'objet père au prototype du fils

- Dynamique
- Héritage via la remontée de la chaîne des prototypes

```
function Voiture() {
  this.demarrer = function() {
    return ”Vroom !" + this.marque;
  }
}

function Sportive(param) {
  this.marque = param;
}
Sportive.prototype = new Voiture();
Sportive.prototype.constructor = Voiture;
Sportive.prototype.accelerer = function() {
  return “Vroom vroom !!”;
}

var maVoiture = new Sportive("Porsche");

Voiture.prototype.freiner = function() {
  return "Hiiin !";
}

//maVoiture.demarrer() retourne : Vroom ! Porsche
//maVoiture.accelerer() retourne : Vroom vroom !!
//maVoiture.freiner() retourne : Hiiin !
```

Aspects techniques méconnus

Orientation objet – Héritage par classes

Apparition de bibliothèques simulant le modèle de classes
- Extention de l'object “Function” pour y ajouter des méthodes
- Exemples
  - Librairie de Douglas Crockford (http://javascript.crockford.com/inheritance.html)
  - Base (http://dean.edwards.name/base)
  - Prototype (http://www.prototypejs.org)

Future évolution d'ECMAScript 4 (et JavaScript 2)
- Modèle alternatif et optionnel de programmation par classes
- Introduction des notions de types, classes et interfaces
- Fait pour faciliter l'adoption par les développeurs
- Mode strict ou mixte

Aspects techniques méconnus

Orientation objet – Encapsulation (1/2)

Propriétés publiques : les composants (attributs ou méthodes) d'un objet sont, par défaut, accessibles à tous ;

```
function Objet1() {
    this.attribut = "Attribut public";
}
MonObjet1.prototype.fonction = function() {
    return "Fonction publique";
}
var MonObjet1 = new Objet1();

MonObjet1.attribut;   //contient "Attribut public"
MonObjet1.fonction(); //retourne "Fonction publique"
```

Propriétés privées : créés dans le constructeur, accessibles seulement via les méthodes privées de l'objet lui-même ;

```
function Objet2() {
    var attribut = "Attribut privé";
    function fonction() {
        return "Fonction privée";
    }
}
var MonObjet2 = new Objet2();

MonObjet2.attribut;   //contient "undefined"
MonObjet2.fonction(); //retourne l'erreur "MonObjet2.fonction() is not a function"
```

Aspects techniques méconnus

Orientation objet – Encapsulation (2/2)

Méthodes privilégiées : méthodes pouvant accéder aux composants privés de l'objet tout en étant accessibles depuis l'extérieur

```
function Objet3() {
    var attribut = "Attribut privé";
    this.fonction = function() {
        return(attribut);
    };
}

var MonObjet3 = new Objet3();

MonObjet3.fonction(); //retourne "Attribut privé"
```

Aspects techniques méconnus

Typage

JavaScript est dynamiquement typé

Cela ne veut pas dire qu'il n'est pas typé

Types primaires
- Récupérable via l'opérateur typeof
- number, string, object, boolean, function, undefined

Autre manière de voir : constructor.name
- Object : objet générique non typé ;
- Boolean : booléen (vaut true ou false) ;
- Number : nombre entier ou décimal ;
- String : chaîne de caractères ;
- Array : tableau ;
- Function : fonction contenant une portion de code ;
- [NomConstructeur] : type défini par l'utilisateur via une fonction constructeur.

Aspects techniques méconnus

Exceptions

try... catch... finally

```
try {
  // Code à tester
  throw "Erreur 1";
} catch(erreur) {
  // Code à exécuter en cas d'erreur
  // erreur = “Erreur 1”
} finally {
  // Code à exécuter dans les deux cas
}
```

Possibilité d'empiler les instructions “catch” chez Mozilla

```
try { ... }
catch ( e if e == "InvalidNameException"  ) { ... }
catch ( e if e == "InvalidIdException"    ) { ... }
catch ( e if e == "InvalidEmailException" ) { ... }
catch ( e ) { ... }
```

Aspects techniques méconnus

Closures

Contexte d'exécution de fonctions imbriquées

- fonction contenue accède aux propriétés de la fonction contenante
- même après l'exécution de la fonction contenante

Exemple

```
function CreerActionVoiture(action) {
  return function() {
    return "La voiture est en train de " + action;
  }
}

function Voiture() {
  this.demarrer = CreerActionVoiture("démarrer");
  this.freiner = CreerActionVoiture("freiner");
  this.arreter = CreerActionVoiture("s'arrêter");
}
var maVoiture = new Voiture();

//maVoiture.demarrer() retourne : La voiture est en train de démarrer
//maVoiture.freiner() retourne : La voiture est en train de freiner
//maVoiture.arreter() retourne : La voiture est en train de s'arrêter
```

Attention : on peut parfois créer des closures sans s'en rendre compte...

Aspects techniques méconnus

ECMAScript for XML (E4X)

Nouveau type d'objet : XML

Implémenté par Mozilla et Adobe

Exemple

```
var voitures = new XML();

voitures = <voitures>
    <voiture marque="Renault" modele="Megane"/>
    <voiture marque="Porsche" modele="911"/>
    <voiture marque="Honda" modele="Civic"/>
  </voitures>;

alert(voitures.voiture.(@marque == "Renault").@modele);
//Affiche: Megane

for each(var voiture in voitures..@marque) {
  alert(marque);
}
//Affiche :
//Renault
//Porsche
//Honda
```

Outils libres disponibles

Interpréteurs en ligne de commande (shell)

SpiderMonkey : js (http://developer-stage.mozilla.org/en/docs/Introduction_to_the_JavaScript_shell)

- Mode interactif
- Shebang

```
#!/usr/bin/js -w
var toto = "titi";
print(toto);
```

Rhino (http://www.mozilla.org/rhino/shell.html)

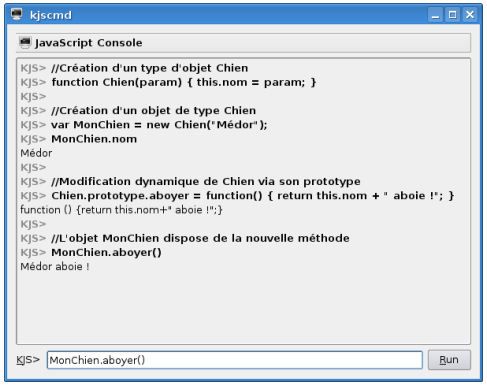
- java org.mozilla.javascript.tools.shell.Main monscript.js

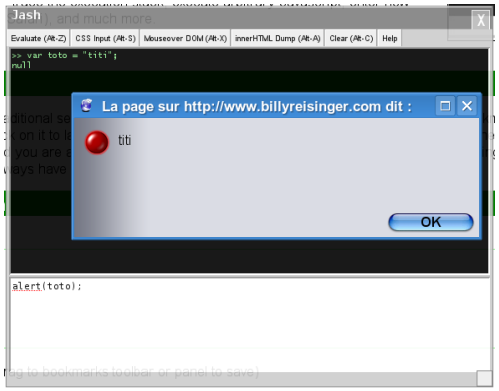
Kjsembed (http://xmelegance.org/kjsembed)

- Script shell KDE wrapper de KJS
- Permet aux applications KDE d'executer du JavaScript
- Kjscmd : ligne de commande ou console graphique

Shells interactifs utilisable dans le browser

- **Jash** (http://www.billyreisinger.com/jash/)
- **Rainbow9** (http://www.rainbow9.org)





Outils libres disponibles

Debuggers

Rhino (http://www.mozilla.org/rhino/debugger.html)

Pour code executé par Rhino (pas browser)

Venkman (https://addons.mozilla.org/fr/firefox/addon/216)

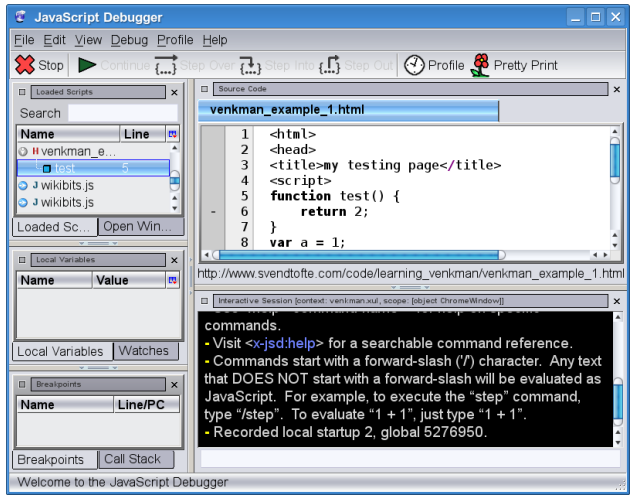
Extension Firefox développée par Mozilla

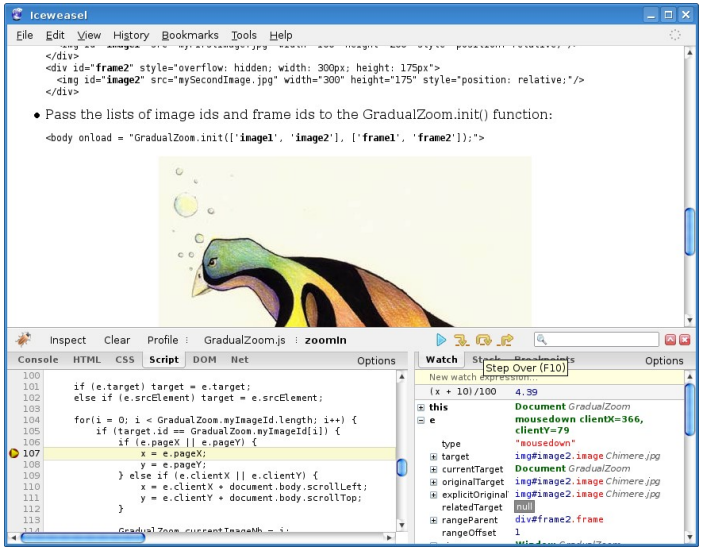
Firebug (http://www.getfirebug.com)

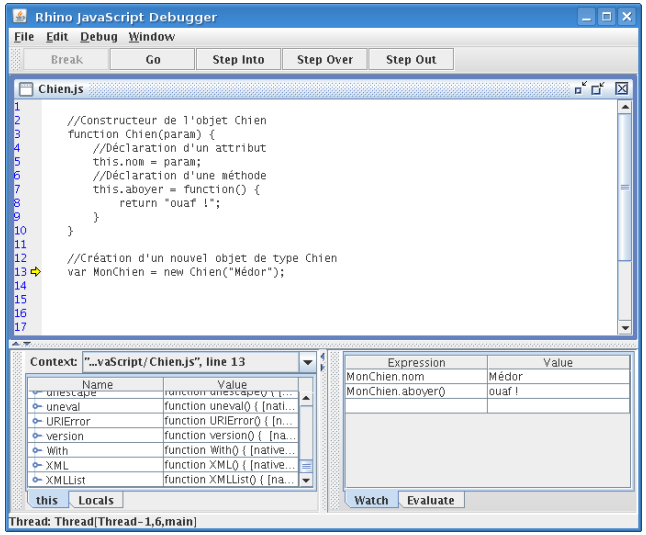
Extension Firefox développée par Joe Hewitt

Debugger mais aussi profiling, logging

Outil très complet (HTML, DOM, CSS)



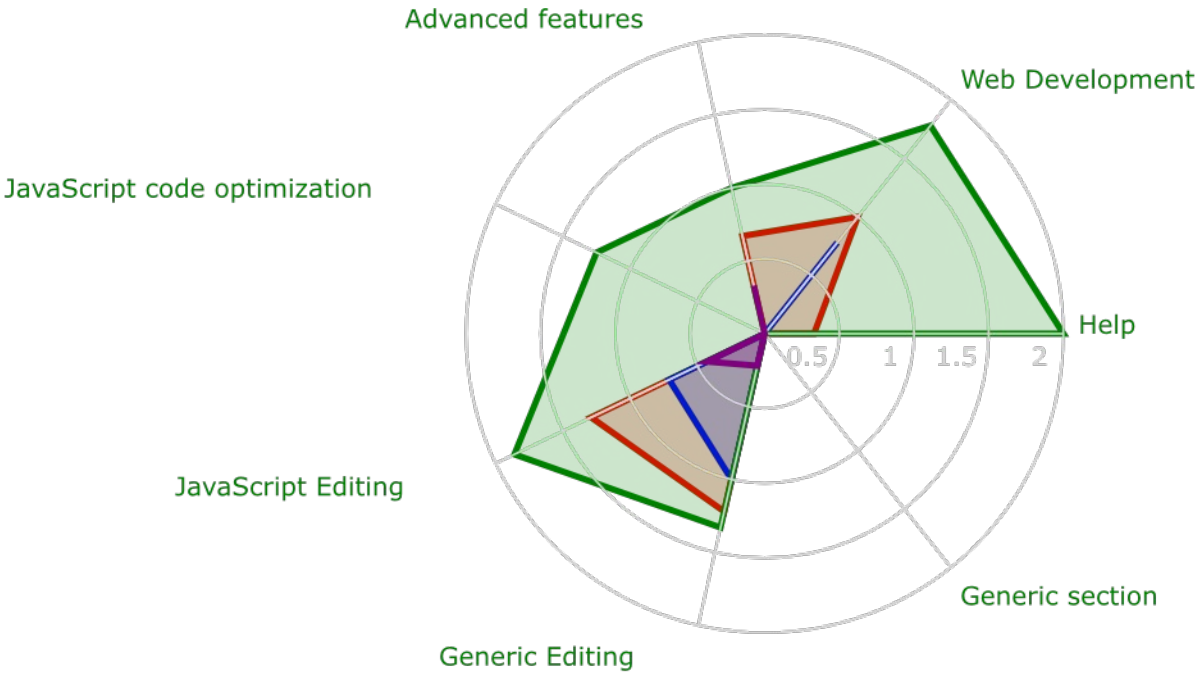




ADVANCE YOUR BUSINESS »

Outils libres disponibles

Environnements de développement (1/2)



Adobe JSEclipse Prerelease 2NetBeans JavaScript Editor 0.2Aptana Studio Community Edition 1.0.0JSide 1.0

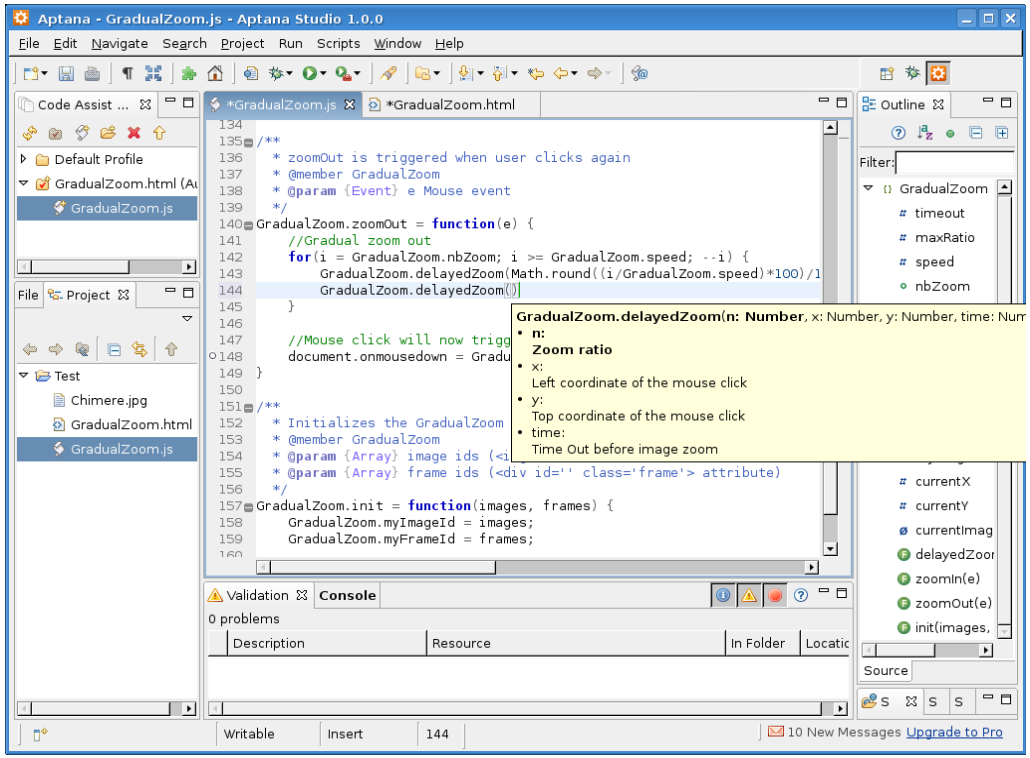
cf. http://www.qsos.org

Outils libres disponibles

Environnements de développement - Aptana (2/2)

Aptana Studio est le plus complet
- Basé sur Eclipse
- Editeur JS avancé
  - Coloration syntaxique
  - Intellisense
  - Autocompletion
- Optimisation de code
  - Logging
  - Debugging
  - Profiling
- Génération JsDoc
- Navigation dans le code

Fonctionnalités HTML, CSS



ADVANCE YOUR BUSINESS »

Outils libres disponibles

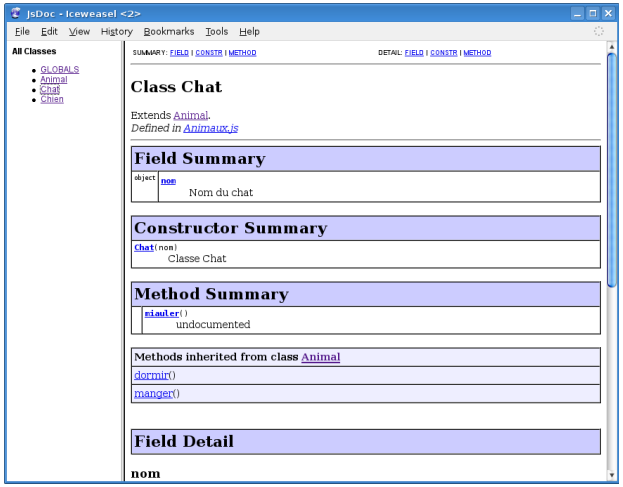
Utilitaires

JSLint (http://www.jslint.com/lint.html)
- JavaScript Verifier
- Plus strict qu'ECMAScript (vision de Crockford)
- Utilisable online, avec Rhino ou WSH

JsUnit (http://www.jsunit.net)
- Tests unitaires
- Il existe une version serveur (Java, Ant)

JSCoverage
- Mesures de couverture de code
- Instrumentation du code
- Rapport dans une page HTML

JsDoc (http://jsdoc.sourceforge.net) et **JsToolkit** (http://jsdoctoolkit.org)
- JavaScript Documentation Tools
- Inspirés de JavaDoc
- JsToolkit est basé sur JsDoc



ADVANCE YOUR BUSINESS »

Le futur de JavaScript

ECMAScript 4

La version 4 de norme ECMAScript est en cours d'élaboration

Elle prend en compte certaines fonctionnalités d'ActionScript 3.0

Nouveautés :

- Orientation objet : modèle plus classique de classes, interfaces
- Modularisation du code : espaces de nommage, notion de packages
- Typage :
  - typage statique optionnel sous forme d'annotations
  - possibilité de mixer des styles de typages
  - mode optionnel de compilation stricte
- Nombreuses autres nouveautés
- ...

cf. http://www.ecmascript.org

Le futur de JavaScript

Le projet Tamarin

Adobe ActionScript Virtual Machine (AVM2) intégrée dans Flash 9
- Machine virtuelle
- Compilateur JIT
- Garbage collector

Don en Novembre 2006 à Mozilla au sein du projet Tamarin
- Implémentation compatible avec ECMAScript 4
- Intégration avec SpiderMonkey (JavaScript 2)

Courant 2008...



Atos Origin

WORLDWIDE IT PARTNER

Questions / Réponses

raphael.semeteys@atosorigin.com

© 2008 Atos Origin. Document exclusivement réservé à usage commercial. Tout ou partie de ce document ne peut être copié, modifié, diffusé ou annoté sans accord écrit d'Atos Origin ou du client.